

FTC ROBOTICS TRAINING SERIES

Teaching the Basic Two-Motor TeleOp

Seven video-guided lessons with teacher pause questions, hands-on practice, safety gates, and technical notes

7 LESSONS	≈2 MIN EACH VIDEO	50 MIN EACH CLASS	1 ROUTINE POWERING ON
---------------------	-----------------------------	-----------------------------	---------------------------------

SERIES OUTCOME Students trace the full TeleOp lifecycle—from hardware mapping and INIT through joystick input, motor commands, telemetry, and split arcade drive—while consistently using the team’s power-on safety sequence.

How to use this guide

- Preview the lesson’s pause table and prepare the listed materials before class.
- Play the video from the beginning. Pause at each exact timestamp while the matching whiteboard visual remains visible.
- Ask the teacher pause question first. Use the Listen for column only after students have reasoned aloud.
- Resume at the same timestamp and continue to the next pause.
- Move into the hands-on task. Enforce the safety gate before every live-motor test.
- Close with the exit ticket; use responses to decide whether to reteach before the next lesson.

Series at a glance

LESSON	CORE CONCEPT	STUDENT PRODUCT
1	How an FTC TeleOp Program Works	Ordered human program trace
2	Meet the Motors: From Java Names to Real Hardware	Verified motor-name/direction record
3	INIT, Telemetry, and waitForStart()	INIT-PLAY-STOP observation

LESSON	CORE CONCEPT	STUDENT PRODUCT
4	The Active Loop: Check, Repeat, Respond	Traced control-loop cycle
5	Reading the Joysticks: Values, Variables, and the Minus Sign	Negated joystick-value predictions
6	Motor Power, Movement, and Telemetry	Motor-power prediction and telemetry check
7	Split Arcade Drive: One Stick Drives, One Stick Turns	Calculated and normalized arcade mix

Course setup and shared materials

DISPLAY	Seven completed training videos; projector or large display with visible playback time
CODE	BasicTwoMotorTeleOp.java plus the split arcade drive version/snippet
ROBOT	Two-motor FTC/goBILDA drivetrain, Robot Controller, Driver Station, and gamepad1
SAFETY	Stable robot blocks/stand, marked safety boundary, ANSI Z87.1 eye protection
CLASSROOM	Printed code, role cards, value cards, observation sheets, pencils, whiteboard/markers

Suggested teaching rhythm

0-5 MIN	Launch: retrieve the previous idea and state today's prediction challenge.
5-20 MIN	Guided video: pause at the listed timestamps; require students to explain evidence from the visual/code.
20-42 MIN	Hands-on: begin unplugged, then use low-power live testing only when the lesson calls for it.
42-47 MIN	Debrief: compare predictions, code values, telemetry, and observed movement.
47-50 MIN	Exit ticket: one short written explanation or calculation.

The non-negotiable POWERING ON sequence

TEACHER DIRECTION Stop any live activity if the call-and-clear sequence is skipped. The operator must restart the sequence from the beginning.

STEP	ACTION	TEACHER CHECK
1	Secure and disable	For first tests, place the robot on stable blocks with wheels clear. Confirm the Driver Station shows the robot disabled.
2	Look around	Inspect the robot and test area. Make sure hands, hair, clothing, tools, wires, and people are clear of moving parts.

STEP	ACTION	TEACHER CHECK
3	Call loudly	The operator faces the team and calls: "POWERING ON!" loudly enough for everyone nearby to hear.
4	Wait for movement	Give teammates time to step away and move behind the safety boundary. Never assume they heard or saw the robot.
5	Enable only when clear	Press PLAY only after the area is visibly clear. Begin with one low-power command at a time.
6	Stop before contact	Press STOP and confirm DISABLED before anyone touches the robot, changes wiring, or adjusts a mechanism.

Required student PPE and readiness: ANSI Z87.1 safety glasses with side shields, closed-toe shoes, long hair tied back, and loose clothing, lanyards, hoodie strings, and jewelry secured.

Teacher safety language

BEFORE PLAY "Robot is on blocks and disabled. Look around. Everyone clear? POWERING ON! ... Wait ... Area is clear. Enabling now."

BEFORE CONTACT "STOP pressed. Driver Station shows disabled. Hands may approach only after the operator confirms disabled."

LESSON 1 • VIDEO 1

How an FTC TeleOp Program Works

BIG IDEA A TeleOp program follows a safe path: identify the OpMode, connect hardware, wait for PLAY, then repeat input-output work until STOP.

CLASS TIME	50 minutes total: 5 min launch • 15 min guided video and discussion • 22 min hands-on • 5 min debrief • 3 min exit ticket
VIDEO	Approximately 2:00; planned pauses expand viewing to about 15 minutes
VOCABULARY	OpMode, LinearOpMode, hardwareMap, INIT, waitForStart(), control loop, telemetry
EVIDENCE	Students can narrate the complete program path and correctly stop the repeated loop when STOP is shown.

Learning objectives

- Name the program's four main phases.
- Distinguish Java motor names from physical hardware connections.
- Trace the repeating control loop from joystick input to telemetry.
- State the POWERING ON sequence before enabling a robot.

Before class

- Display or print BasicTwoMotorTeleOp.java.
- Prepare cards labeled IDENTIFY, CONNECT, WAIT, READ, SET POWER, TELEMETRY, REPEAT, and STOP.
- Keep the robot disabled; this lesson's activity can be completed without live motor power.

Guided video: pause-and-discuss plan

Pause at the timestamp shown. Keep the whiteboard visual on screen, ask the question, take two or three student responses, then use "Listen for" to clarify—not to turn the pause into a lecture.

PAUSE	TEACHER PAUSE QUESTION	LISTEN FOR	THEN
0:18	What are the program's four main stations?	Identify; connect; wait safely for PLAY; repeat the control loop.	▶ 0:18
0:47	Do leftMotor and rightMotor already control physical motors after they are declared?	No. They are Java fields/name tags until hardwareMap assigns configured devices.	▶ 0:47

PAUSE	TEACHER PAUSE QUESTION	LISTEN FOR	THEN
0:58	Why must the Robot Configuration names match the Java strings exactly?	The quoted strings are exact, case-sensitive device identifiers; a mismatch stops initialization.	► 0:58
1:06	What does <code>waitForStart()</code> do?	It pauses the <code>LinearOpMode</code> until <code>PLAY</code> , or returns if <code>STOP</code> is requested.	► 1:06
1:26	Before enabling, what must the operator and team do?	Secure the robot, look around, call “POWERING ON!”, allow time to move, and enable only when clear.	► 1:26
1:37	What four actions repeat inside the control loop?	Read the sticks; set motor power; send telemetry; return and check/repeat.	► 1:37
1:48	Why does the joystick code contain a minus sign?	Forward Y is normally negative; negation makes a forward command positive.	► 1:48

Hands-on: Human TeleOp Flow

TIME	20 minutes
SUCCESS	Students can narrate the complete program path and correctly stop the repeated loop when <code>STOP</code> is shown.

1. Place the phase cards in a scrambled pile. Ask teams to arrange the startup and control sequence.
2. Give one student a program token. Move it through `IDENTIFY` → `CONNECT` → `WAIT`.
3. At `WAIT`, require a second student to act as the Driver Station and say whether `INIT`, `PLAY`, or `STOP` was pressed.
4. After `PLAY`, circulate the token through `READ` → `SET POWER` → `TELEMETRY` → `REPEAT` three times.
5. Change a joystick-value card on each cycle so students see that the loop uses fresh input.
6. Show the `STOP` card. Students must end the loop and place the token at `DISABLED`.
7. Finish with a whole-class rehearsal of the `POWERING ON` call-and-clear sequence without enabling the robot.

SAFETY GATE Before any live-motor step: robot on blocks, disabled, area checked, “POWERING ON!” called loudly, time allowed to step clear, then `PLAY`. `STOP` and confirm `DISABLED` before contact.

Teacher technical notes

- The supplied program uses tank drive: each joystick controls one drivetrain side independently.

- The outer if (opModelsActive()) is redundant because the while loop checks the same condition. Keep it for this walkthrough and refactor later.
- The annotation group says “Iterative Opmode,” but the class extends LinearOpMode. The group is only a Driver Station organization label.
- The motor-direction commands are commented out. Confirm mounting and configure the necessary side as REVERSE before a floor test.

Debrief and exit ticket

EXIT PROMPT In one sentence, describe how a joystick movement eventually becomes a motor command.

LESSON 2 • VIDEO 2

Meet the Motors: From Java Names to Real Hardware

BIG IDEA Declaring a DcMotor creates a Java reference; hardwareMap connects that reference to an exactly named configured device.

CLASS TIME	50 minutes total: 5 min launch • 15 min guided video and discussion • 22 min hands-on • 5 min debrief • 3 min exit ticket
VIDEO	Approximately 2:00; planned pauses expand viewing to about 15 minutes
VOCABULARY	field, DcMotor, variable name, hardwareMap.get(), class literal, configuration name, Direction.REVERSE
EVIDENCE	The code names match the configuration exactly, and both wheel directions match the team's forward convention at low power.

Learning objectives

- Explain the difference between declaring and mapping a motor.
- Identify the two arguments in hardwareMap.get().
- Detect a configuration-name mismatch.
- Perform a safe first direction test.

Before class

- Open the Robot Configuration and the Java file side by side.
- Secure the robot on blocks so both wheels are fully clear.
- Prepare a direction-observation sheet with left wheel and right wheel columns.

Guided video: pause-and-discuss plan

Pause at the timestamp shown. Keep the whiteboard visual on screen, ask the question, take two or three student responses, then use “Listen for” to clarify—not to turn the pause into a lecture.

PAUSE	TEACHER PAUSE QUESTION	LISTEN FOR	THEN
0:25	What does declaring a variable accomplish?	It creates a typed Java reference/name that can hold a DcMotor object.	► 0:25

PAUSE	TEACHER PAUSE QUESTION	LISTEN FOR	THEN
0:34	Does declaring leftMotor connect it to a physical motor?	No. The reference is not mapped until hardwareMap.get() runs.	► 0:34
0:45	What job does hardwareMap perform?	It is the Robot Controller's directory of configured hardware devices.	► 0:45
0:56	What are the two important arguments inside hardwareMap.get()?	The device type, DcMotor.class, and the exact configuration-name string.	► 0:56
1:06	Why must left_motor include the underscore?	The configuration string must match every character and capital exactly.	► 1:06
1:23	Why might one drivetrain motor need to be reversed?	Opposite-side motors are often mirrored, so identical shaft directions can move wheels oppositely.	► 1:23
1:42	What is the safe sequence for the first direction test?	Blocks and disabled; inspect and clear; call out; wait; enable; use low input; STOP and disable before changes.	► 1:42

Hands-on: Map and Verify the Motors

TIME	25 minutes
SUCCESS	The code names match the configuration exactly, and both wheel directions match the team's forward convention at low power.

1. With the robot disabled, locate the two motor names in Robot Configuration.
2. Compare the names character by character with the strings in hardwareMap.get(). Circle any difference.
3. Ask students to predict the result of left_motor being misspelled in code.
4. Confirm the robot is stable on blocks and both wheels are clear.
5. Follow the POWERING ON procedure. Move each joystick gently and record the corresponding wheel direction.
6. If a direction is wrong, press STOP, confirm DISABLED, and then discuss which direction setting may need changing.
7. Repeat the low-power test after the code/configuration change; never touch the drivetrain while enabled.

SAFETY GATE Before any live-motor step: robot on blocks, disabled, area checked, "POWERING ON!" called loudly, time allowed to step clear, then PLAY. STOP and confirm DISABLED before contact.

Teacher technical notes

- `private DcMotor leftMotor;` declares a field; it does not instantiate or map a motor.
- `hardwareMap.get(DcMotor.class, "left_motor")` returns the configured `DcMotor` associated with that exact name.
- FTC configuration names are case-sensitive strings and should be treated as exact identifiers.
- Commented direction lines have no effect until the leading `//` is removed.
- The reversed side depends on mounting, gearing, and the team's forward convention; the right side is not universally reversed.
- Stop and disable before students touch the drivetrain or change physical connections.

Debrief and exit ticket

EXIT PROMPT Complete the analogy: declaring a motor is making a name tag; `hardwareMap` is ____.

LESSON 3 • VIDEO 3

INIT, Telemetry, and waitForStart()

BIG IDEA INIT performs setup and reaches a safe waiting point; PLAY releases the program into active control; STOP ends it.

CLASS TIME	50 minutes total: 5 min launch • 15 min guided video and discussion • 22 min hands-on • 5 min debrief • 3 min exit ticket
VIDEO	Approximately 2:00; planned pauses expand viewing to about 15 minutes
VOCABULARY	INIT, PLAY, STOP, runOpMode(), telemetry packet, waitForStart(), opModelsActive()
EVIDENCE	Students correctly predict the robot's state and code location during INIT, after PLAY, and after STOP.

Learning objectives

- Describe what happens when INIT is pressed.
- Differentiate telemetry.addData() from telemetry.update().
- Explain how waitForStart() pauses a LinearOpMode.
- Describe the effects of PLAY and STOP.

Before class

- Put the robot on blocks and keep it disabled.
- Open the Driver Station to Basic Two Motor TeleOp.
- Ensure the Status telemetry line is visible in the code.

Guided video: pause-and-discuss plan

Pause at the timestamp shown. Keep the whiteboard visual on screen, ask the question, take two or three student responses, then use “Listen for” to clarify—not to turn the pause into a lecture.

PAUSE	TEACHER PAUSE QUESTION	LISTEN FOR	THEN
0:17	What causes runOpMode() to begin executing?	Selecting the OpMode and pressing INIT causes the SDK to invoke it.	► 0:17
0:27	What setup work happens before waitForStart()?	Hardware is mapped and initialization telemetry is prepared/sent.	► 0:27
0:46	What is the difference between addData() and update()?	addData prepares an item; update transmits the current packet to the Driver Station.	► 0:46

PAUSE	TEACHER PAUSE QUESTION	LISTEN FOR	THEN
0:55	Does “Initialized” mean the driving loop is running?	No. It means preceding setup code ran; the program is still waiting for PLAY.	► 0:55
1:05	What does waitForStart() wait for?	PLAY, unless STOP is requested first.	► 1:05
1:34	What changes after PLAY is pressed?	The wait gate opens, the active check can be true, and control-loop code can execute.	► 1:34
1:53	What happens to opModelsActive() after STOP?	It becomes false; the loop exits and the runtime disables outputs.	► 1:53

Hands-on: Observe INIT → PLAY → STOP

TIME	25 minutes
SUCCESS	Students correctly predict the robot’s state and code location during INIT, after PLAY, and after STOP.

1. Secure the robot on blocks and confirm it is disabled.
2. Select Basic Two Motor TeleOp and press INIT.
3. Locate Status: Initialized on the Driver Station and identify the code that produced it.
4. Move the sticks before PLAY and confirm the motors do not respond.
5. Point to waitForStart() and explain that execution is blocked there.
6. Follow the POWERING ON and step-clear procedure, then press PLAY.
7. Move one stick gently, press STOP, and confirm the wheel stops and the OpMode ends.

SAFETY GATE Before any live-motor step: robot on blocks, disabled, area checked, “POWERING ON!” called loudly, time allowed to step clear, then PLAY. STOP and confirm DISABLED before contact.

Teacher technical notes

- In a LinearOpMode, the SDK invokes runOpMode() during initialization; execution proceeds until waitForStart().
- waitForStart() blocks until the OpMode starts or a stop is requested.
- telemetry.addData() adds an item to the packet; telemetry.update() transmits current telemetry.
- The initialized message confirms preceding code ran; it does not prove that every mechanism is safe or correctly oriented.
- After START, the outer active check determines whether the supplied program enters the loop.
- After STOP, opModelsActive() is false and the runtime disables hardware outputs.

Debrief and exit ticket

EXIT PROMPT Why can “Status: Initialized” be true while the wheels still do not respond to a joystick?

LESSON 4 • VIDEO 4

The Active Loop: Check, Repeat, Respond

BIG IDEA The while loop repeatedly checks the OpMode state, reads fresh input, commands hardware, and reports telemetry.

CLASS TIME	50 minutes total: 5 min launch • 15 min guided video and discussion • 22 min hands-on • 5 min debrief • 3 min exit ticket
VIDEO	Approximately 2:00; planned pauses expand viewing to about 15 minutes
VOCABULARY	boolean, true, false, if, while, condition, braces, iteration, loop body
EVIDENCE	Students can trace a loop cycle in order and explain why FALSE prevents another iteration.

Learning objectives

- Explain what `opModelsActive()` checks.
- Compare the outer if with the inner while.
- Identify the loop body and trace one cycle.
- Explain why continuous input sampling makes TeleOp responsive.

Before class

- Print the code with enough space for students to outline braces.
- Prepare role cards: CHECK, READ, COMMAND, REPORT.
- Prepare TRUE/FALSE and several joystick-value cards.

Guided video: pause-and-discuss plan

Pause at the timestamp shown. Keep the whiteboard visual on screen, ask the question, take two or three student responses, then use “Listen for” to clarify—not to turn the pause into a lecture.

PAUSE	TEACHER PAUSE QUESTION	LISTEN FOR	THEN
0:17	What two values can <code>opModelsActive()</code> return?	true or false.	► 0:17
0:26	How often does the outer if perform its check?	Once when execution reaches the if.	► 0:26
0:36	How often does the while condition perform its check?	Before every attempted loop iteration.	► 0:36

PAUSE	TEACHER PAUSE QUESTION	LISTEN FOR	THEN
0:45	What do the braces identify?	The exact block of statements that belongs to and repeats with the loop.	► 0:45
1:05	What three major jobs happen inside each cycle?	Read the controls, command motor power, and report telemetry.	► 1:05
1:44	Why read the joysticks again every cycle?	To respond to the driver's newest position rather than reuse an old command.	► 1:44
1:53	What causes the loop to exit?	STOP or another stop request makes <code>opModelsActive()</code> false.	► 1:53

Hands-on: Be the Control Loop

TIME	22 minutes
SUCCESS	Students can trace a loop cycle in order and explain why FALSE prevents another iteration.

1. Students outline the opening and closing braces of the while loop on the printed code.
2. Assign four students the roles CHECK, READ, COMMAND, and REPORT.
3. Pass a program token through one complete cycle using a joystick-value card.
4. At CHECK, show TRUE and repeat with a different input card for forward, turn, and stop.
5. Ask observers to point to the code line represented by each role.
6. Show FALSE to simulate STOP; the token must leave the loop immediately.
7. Transfer the idea to the secured robot using the POWERING ON procedure and observe fresh telemetry values.

SAFETY GATE Before any live-motor step: robot on blocks, disabled, area checked, "POWERING ON!" called loudly, time allowed to step clear, then PLAY. STOP and confirm DISABLED before contact.

Teacher technical notes

- `opModelsActive()` is true only after START and while a stop has not been requested.
- An if condition is evaluated once when reached; a while condition is evaluated before every attempted iteration.
- The outer if is redundant because the while performs the same check. Preserve it for this source walkthrough and simplify later.
- The loop has no explicit delay and repeats as quickly as the runtime and its operations permit.

- Fresh gamepad values on each cycle make driver control responsive.
- After STOP, the loop exits and the FTC runtime disables hardware outputs.

Debrief and exit ticket

EXIT PROMPT What would the robot feel like to drive if the joysticks were read only once? Explain.

LESSON 5 • VIDEO 5

Reading the Joysticks: Values, Variables, and the Minus Sign

BIG IDEA Each loop reads floating-point gamepad values; the Y-axis minus sign converts FTC's negative-forward convention into positive forward power.

CLASS TIME	50 minutes total: 5 min launch • 15 min guided video and discussion • 22 min hands-on • 5 min debrief • 3 min exit ticket
VIDEO	Approximately 2:00; planned pauses expand viewing to about 15 minutes
VOCABULARY	double, assignment, expression, gamepad1, left_stick_y, right_stick_y, unary minus, negate
EVIDENCE	Students correctly negate at least four sample values and connect the result to the left or right power variable.

Learning objectives

- Read an assignment statement from right to left.
- Explain why double stores motor-power values.
- Trace left and right stick Y inputs.
- Negate sample Y-axis values and predict motion.

Before class

- Prepare RAW VALUE, MINUS SIGN, and POWER VARIABLE cards.
- Prepare number cards: -1.0, -0.7, 0.0, +0.4, and +1.0.
- If testing live, put the robot on blocks and open power telemetry.

Guided video: pause-and-discuss plan

Pause at the timestamp shown. Keep the whiteboard visual on screen, ask the question, take two or three student responses, then use “Listen for” to clarify—not to turn the pause into a lecture.

PAUSE	TEACHER PAUSE QUESTION	LISTEN FOR	THEN
0:27	What does the word double tell Java?	The variable stores a floating-point/decimal numeric value.	► 0:27

PAUSE	TEACHER PAUSE QUESTION	LISTEN FOR	THEN
0:36	Which side of an assignment is calculated first?	The expression on the right; its result is stored in the variable on the left.	► 0:36
0:46	What does gamepad1 identify, and what direction does the Y axis measure?	The first controller; the stick's forward-and-backward axis.	► 0:46
0:56	How does tank drive use the two sticks?	Each stick independently supplies the requested power for one drivetrain side.	► 0:56
1:06	What raw value normally represents a fully forward FTC stick?	Approximately -1.0.	► 1:06
1:15	What does the minus sign do to that value?	It negates the sign: $-(-1.0)$ becomes $+1.0$.	► 1:15

Hands-on: The Human Minus Sign

TIME	20 minutes
SUCCESS	Students correctly negate at least four sample values and connect the result to the left or right power variable.

1. Assign three students the roles RAW VALUE, MINUS SIGN, and POWER VARIABLE.
2. Give RAW VALUE the -0.7 card. MINUS SIGN changes its sign, and POWER VARIABLE displays +0.7.
3. Repeat with 0.0, +0.4, and -1.0.
4. After each result, classmates predict forward, stop, or backward under the program's positive-forward convention.
5. Ask students to read `double leftPower = -gamepad1.left_stick_y;` from right to left.
6. Secure the robot on blocks, perform the POWERING ON sequence, and compare stick position with live telemetry.

SAFETY GATE Before any live-motor step: robot on blocks, disabled, area checked, "POWERING ON!" called loudly, time allowed to step clear, then PLAY. STOP and confirm DISABLED before contact.

Teacher technical notes

- FTC stick axes use floating-point values, typically from -1.0 to +1.0.
- Pushing an FTC Y axis forward normally produces a negative raw value; negation makes the program's forward command positive.
- The unary minus applies before the result is assigned to the variable.
- `leftPower` and `rightPower` are local variables recreated during every loop iteration in this program.

- A real gamepad may not return exactly zero when released. A deadband is a useful later improvement.
- Input negation sets a convenient convention; physical wheel direction still depends on motor configuration and mounting.

Debrief and exit ticket

EXIT PROMPT If `left_stick_y` is +0.35, what value is stored in `leftPower`, and what direction is requested?

LESSON 6 • VIDEO 6

Motor Power, Movement, and Telemetry

BIG IDEA `setPower()` turns computed values into motor commands, while telemetry lets the team compare the command with observed behavior.

CLASS TIME	50 minutes total: 5 min launch • 15 min guided video and discussion • 22 min hands-on • 5 min debrief • 3 min exit ticket
VIDEO	Approximately 2:00; planned pauses expand viewing to about 15 minutes
VOCABULARY	object, method, argument, <code>setPower()</code> , magnitude, sign, telemetry, commanded value
EVIDENCE	Students predict each low-power test, identify the matching telemetry, and stop safely on any mismatch.

Learning objectives

- Identify the object, method, and argument in `setPower()`.
- Interpret sign and magnitude across the motor-power range.
- Predict basic tank-drive movement from two power values.
- Use telemetry in a safe, systematic motor test.

Before class

- Secure the robot on blocks with both wheels clear.
- Prepare prediction cards for 0.0/0.0, +0.3/+0.3, -0.3/-0.3, and +0.3/-0.3.
- Confirm Left Power and Right Power telemetry are visible.

Guided video: pause-and-discuss plan

Pause at the timestamp shown. Keep the whiteboard visual on screen, ask the question, take two or three student responses, then use “Listen for” to clarify—not to turn the pause into a lecture.

PAUSE	TEACHER PAUSE QUESTION	LISTEN FOR	THEN
0:28	What are the object, method, and argument in <code>leftMotor.setPower(leftPower)</code> ?	Object: <code>leftMotor</code> ; method: <code>setPower</code> ; argument: <code>leftPower</code> .	► 0:28
0:38	What requests stop? What do magnitude and sign communicate?	0.0 requests stop; magnitude is how much output; sign is commanded direction relative to configuration.	► 0:38
0:48	What happens when both sides receive equal positive values?	They should drive straight forward if directions and mechanics are correct.	► 0:48

PAUSE	TEACHER PAUSE QUESTION	LISTEN FOR	THEN
0:58	What happens with opposite signs?	The sides move oppositely, causing an in-place turn or spin.	▶ 0:58
1:08	What does telemetry help the team compare?	The program's commanded values with the observed direction/behavior.	▶ 1:08
1:47	What should the driver do if movement differs from the prediction?	Press STOP immediately, confirm disabled, then diagnose one item at a time.	▶ 1:47

Hands-on: Predict, Command, Observe, Verify

TIME	25 minutes
SUCCESS	Students predict each low-power test, identify the matching telemetry, and stop safely on any mismatch.

1. Before enabling, students predict the behavior for each prepared left/right value pair.
2. Confirm the robot is stable on blocks and the wheel paths are clear.
3. Perform the full POWERING ON call-and-clear sequence.
4. Move the left stick alone, return it to center, then move the right stick alone.
5. Move both sticks gently and compare telemetry with wheel direction.
6. Press STOP immediately if observed motion differs from the prediction.
7. Confirm DISABLED before checking direction settings, configuration names, wiring, or mechanical interference.

SAFETY GATE Before any live-motor step: robot on blocks, disabled, area checked, "POWERING ON!" called loudly, time allowed to step clear, then PLAY. STOP and confirm DISABLED before contact.

Teacher technical notes

- `setPower(double power)` accepts a requested power value; usable motor power is normally constrained to -1.0 through +1.0.
- The sign controls commanded direction relative to the configured motor direction; mounting also affects physical wheel motion.
- Equal values produce straight motion only when the drivetrain is aligned and directions are configured correctly.
- Telemetry reports commanded program values, not independent measurements of actual wheel speed.

- Use low input values on blocks for the first test.
- The runtime disables outputs after STOP, but students must still confirm DISABLED before touching mechanisms.

Debrief and exit ticket

EXIT PROMPT Why can telemetry show +0.3 while a wheel still turns the wrong physical direction?

LESSON 7 • VIDEO 7

Split Arcade Drive: One Stick Drives, One Stick Turns

BIG IDEA Split arcade drive combines a forward/back command and a turn command, then normalizes the mixed outputs so both remain in range.

CLASS TIME	50 minutes total: 5 min launch • 15 min guided video and discussion • 22 min hands-on • 5 min debrief • 3 min exit ticket
VIDEO	Approximately 2:00; planned pauses expand viewing to about 15 minutes
VOCABULARY	split arcade drive, drive, turn, mix, absolute value, denominator, normalize, steering ratio
EVIDENCE	Students compute and normalize at least three drive/turn cases, then connect the math to observed motion and telemetry.

Learning objectives

- Distinguish tank drive from split arcade drive.
- Explain the drive and turn inputs.
- Compute left and right motor mixes.
- Normalize an out-of-range mix and test it safely.

Before class

- Display the split arcade key code.
- Prepare DRIVE, TURN, LEFT POWER, and RIGHT POWER cards plus simple value cards.
- Secure the robot on blocks and show Drive, Turn, Left Power, and Right Power telemetry.

Guided video: pause-and-discuss plan

Pause at the timestamp shown. Keep the whiteboard visual on screen, ask the question, take two or three student responses, then use “Listen for” to clarify—not to turn the pause into a lecture.

PAUSE	TEACHER PAUSE QUESTION	LISTEN FOR	THEN
0:18	What is the difference between tank drive and split arcade drive?	Tank assigns one stick to each side; split arcade uses left Y for forward/back and right X for turning.	► 0:18

PAUSE	TEACHER PAUSE QUESTION	LISTEN FOR	THEN
0:28	Which axis supplies drive, and why is it negated?	gamepad1.left_stick_y; negation makes forward input positive.	▶ 0:28
0:38	Which axis supplies turn?	gamepad1.right_stick_x.	▶ 0:38
0:48	Why does one formula add turn while the other subtracts it?	Turning requires increasing one side relative to the other; opposite turn contributions create steering.	▶ 0:48
1:08	How can mixed values exceed the motor-power range?	drive and turn can each approach 1.0, so adding them can produce a magnitude as large as 2.0.	▶ 1:08
1:18	What does normalization accomplish?	It scales both outputs together into range without changing their steering ratio.	▶ 1:18

Hands-on: Mix and Normalize

TIME	25 minutes
SUCCESS	Students compute and normalize at least three drive/turn cases, then connect the math to observed motion and telemetry.

1. Assign students DRIVE, TURN, LEFT POWER, and RIGHT POWER roles.
2. Use drive = +0.5 and turn = 0.0; compute both motor powers and predict motion.
3. Use drive = 0.0 and turn = +0.5; compute the outputs and predict the turn.
4. Use drive = +1.0 and turn = +1.0; identify the raw +2.0/0.0 result as out of range.
5. Calculate denominator = 2.0 and normalize to +1.0/0.0. Emphasize that both sides use the same divisor.
6. Secure the robot on blocks and follow the POWERING ON procedure.
7. Test forward, backward, right turn, left turn, and gentle combined commands one at a time. STOP on any mismatch.

SAFETY GATE Before any live-motor step: robot on blocks, disabled, area checked, "POWERING ON!" called loudly, time allowed to step clear, then PLAY. STOP and confirm DISABLED before contact.

Key code for the board

```
double drive = -gamepad1.left_stick_y;  
double turn = gamepad1.right_stick_x;  
double denominator = Math.max(Math.abs(drive) + Math.abs(turn), 1.0);  
double leftPower = (drive + turn) / denominator;  
double rightPower = (drive - turn) / denominator;
```

Teacher technical notes

- This is split arcade or two-stick arcade drive. Single-stick arcade uses the X and Y axes of one stick.
- With positive motor power configured forward, positive turn produces positive left and negative right contributions, turning right.
- $\text{Math.max}(\text{Math.abs}(\text{drive}) + \text{Math.abs}(\text{turn}), 1.0)$ keeps the denominator at least 1.0, so ordinary values are not amplified.
- When the raw mix exceeds magnitude 1.0, dividing both sides by the same denominator preserves the steering ratio.
- The companion example reverses the right motor, but the required side depends on the physical drivetrain.
- A joystick deadband and response shaping are useful later improvements, outside this introductory lesson.

Debrief and exit ticket

EXIT PROMPT For drive = +0.6 and turn = +0.2, calculate leftPower and rightPower. Is normalization needed?

SERIES CLOSING

Build smart. Speak up. Stay safe.

Students are ready to move from reading a simple TeleOp to testing, explaining, and improving it. Keep the next improvement small and observable: add a deadband, shape joystick response, simplify the redundant active check, or compare tank and split arcade drive on a controlled course.

FINAL TEACHER CHECK Students should be able to predict before they enable, compare telemetry while they test, and press STOP before they troubleshoot. Those habits matter as much as the code.